Evaluating Inductive Bias and Sample Efficiency: A Comparative Study of CNNs and Vision Transformers on Sign Language MNIST

Hansini Rajesh (hr394)
Ishaan Agarwal (ia299)

March 18, 2026

Abstract

This study evaluates the architectural trade-offs between Convolutional Neural Networks (CNNs) and Vision Transformers (ViTs) using the Sign Language MNIST dataset. By training a lightweight ResNet and a Minimal ViT on progressive data splits (1% to 100%), we quantify how inherent spatial inductive biases affect sample efficiency. Key findings reveal that the CNN consistently outperforms the ViT in data-starved regimes, achieving 84.55% test accuracy with only 5% of the data. While the ViT reached 95.66% accuracy at full scale with data augmentation, it required significantly more parameters and memory to compensate for its lack of built-in spatial priors. The results demonstrate that traditional convolutional architectures remain the superior choice for small-scale, resource-constrained visual recognition tasks.

1 Introduction

In the landscape of modern computer vision, the pursuit of state-of-the-art performance has increasingly relied on massive datasets and highly parameterized models. While Vision Transformers (ViTs) have recently emerged as powerful alternatives to traditional Convolutional Neural Networks (CNNs), their reliance on global self-attention mechanisms comes with a significant trade-off: a severe lack of inductive bias. This project investigates the behavior of competing neural architectures under a strict data-constrained regime using the Sign Language MNIST dataset.

The primary objective is to evaluate how different models generalize when restricted to a limited sample of low-resolution (28×28) grayscale images. By tracking the evolution of predictive performance from classical spatial models (CNNs, specifically ResNet) to modern sequence-to-sequence visual models (ViTs). This report aims to demonstrate that highly flexible models like Transformers require exponentially more data to learn the spatial priors inherently built into convolutional architectures. Ultimately, this study aims to illustrate why "generic" or traditional models often remain the superior choice in environments where data is finite and computational efficiency is prioritized.

1.1 Problem Statement

The automated recognition of human gestures is a cornerstone of modern Human-Computer Interaction (HCI), providing a vital bridge for inclusive communication. American Sign Language (ASL) is the primary mode of communication for many individuals, yet there remains a significant communication gap between the deaf and hard-of-hearing community and those who do not know the language. In this report we explore the Sign Language MNIST dataset to distinguish between

the traditional CNN models and ViTs, highlighting the strong spatial inductive biases of traditional models, that the transformer models fundamentally lack.

1.2 Dataset Overview

The Sign Language MNIST dataset, can be found on Kaggle, it contains 34,627 images (27,455 training, 7,172 testing). Each observation is a 28x28 grayscale image (784 pixel intensity features, 0-255). The target is a categorical label for 24 letters of the American Sign Language (ASL) alphabet, excluding 'J' and 'Z' due to their required motion. Labels are numerically encoded 0-25 (skipping 9 and 25).

1.3 Project Goals

This project aims to quantify the trade-off between architectural inductive bias and data volume through the following targeted experiments:

1. Evaluate Sample Efficiency via Data Starvation: Train both the CNN and ViT on progressively smaller subsets of the training data (e.g., 100%, 50%, 25%, 10%, 5%, and 1%) to plot a comprehensive generalization curve for each architecture.

2. Assess the Impact of Augmentation: Apply data augmentation to the constrained subsets to test if artificially increasing data diversity can compensate for the ViT's lack of inherent spatial assumptions.

3. Analyze Computational Trade-offs: Track training time, memory footprint, and inference speed across the different dataset sizes to evaluate the practical efficiency and scalability of both models.

2 Exploratory Data Analysis

In this section, we try to explore the data and understand some of the patterns in the dataset.

The dataset contains 24 distinct classes representing the American Sign Language (ASL) alphabet, explicitly excluding the letters 'J' and 'Z', as their gestures require dynamic motion that cannot be captured in static 2D images. We can see a sample of the dataset in Figure 1(a).

Analysis of the global class distribution reveals a relatively balanced dataset, with each class containing approximately 1,000 to 1,200 training instances. Figure 1(b) shows the class distribution of the various letters in the dataset.

Figure 2 visualizes the pixel-wise variance (standard deviation) for each sign language class. The heatmaps reveal that the highest variance. The plot shows that the most discriminative information is heavily concentrated along the contours and edges of the hands.

This finding strongly supports our hypothesis regarding architectural inductive bias. CNNs can exploit their inherent edge-detecting priors, resulting in high sample efficiency. In contrast, Vision Transformers (ViTs) lack these spatial priors and need significant data to learn these boundary relationships from scratch.

3 Data Preprocessing

3.1 Cleaning

- **Missing Values:** The dataset was strictly validated, containing no missing or null values.

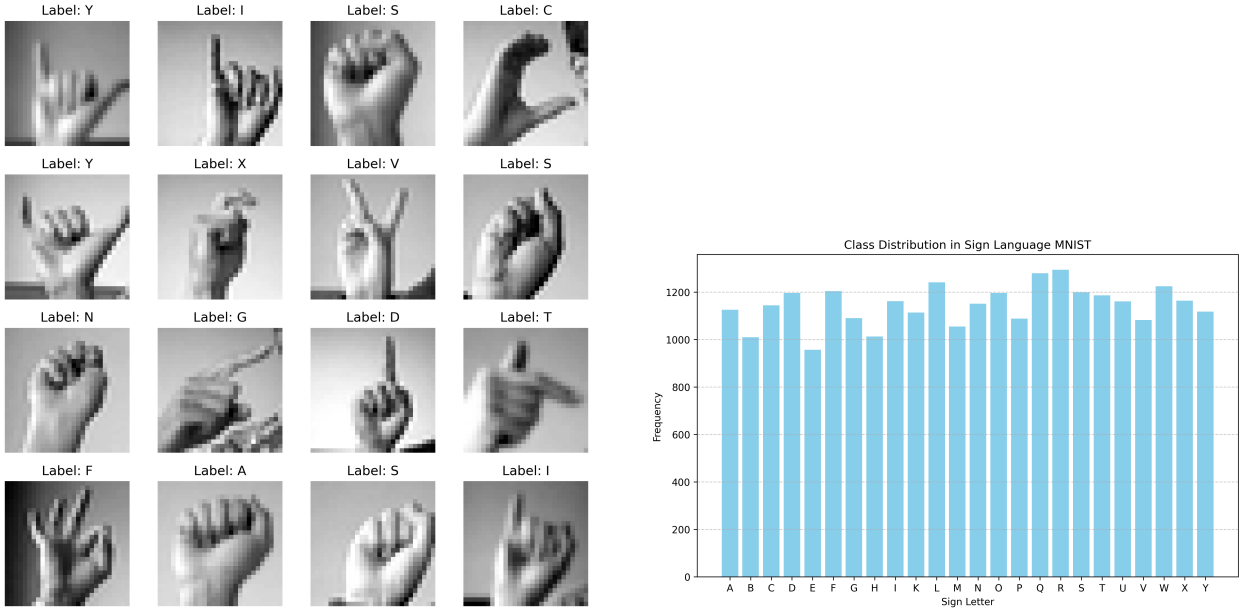


Figure 1: Overview of the dataset: Sample images with labels (left) and class distribution (right).

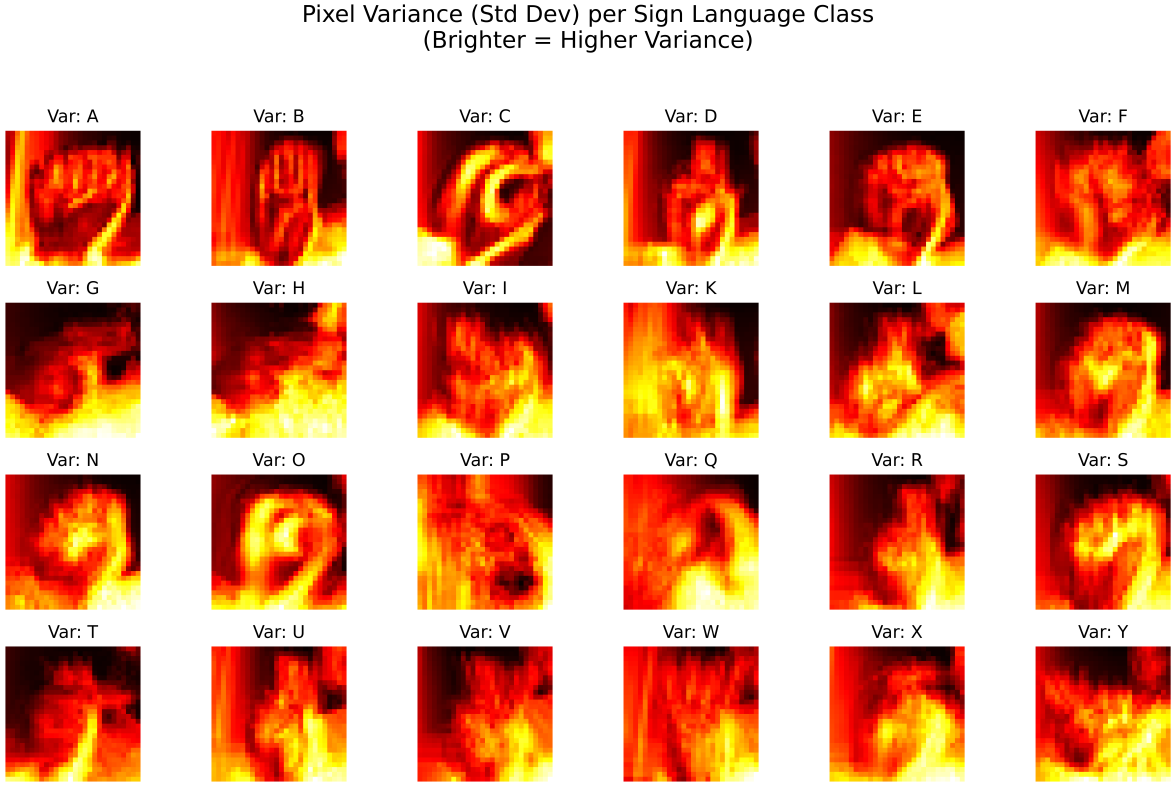


Figure 2: Pixel-wise variance (standard deviation) heatmaps for each ASL class. The concentration of high variance along the contours and edges of the hand gestures supports the hypothesis that convolutional architectures, with their inherent edge-detecting spatial priors, are more sample-efficient than Transformers in this domain

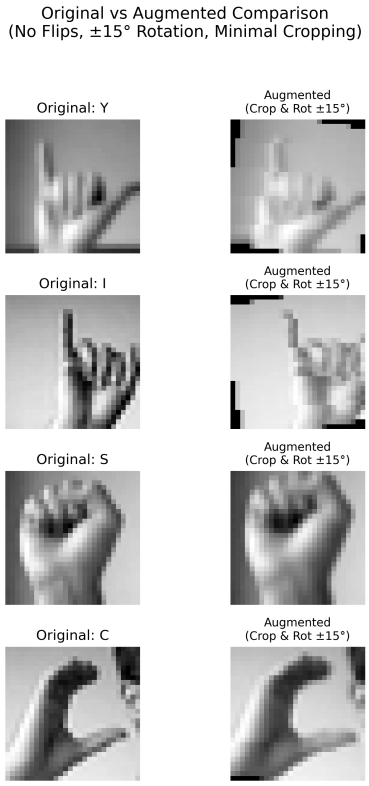


Figure 3: Visual representation of data augmentation techniques.

- **Deduplication & Outliers:** No duplicate removal was required, and the target labels omitted the motion-dependent letters 'J' and 'Z'.

3.2 Feature Engineering and Transformations

To prepare the data for the CNN and ViT architectures, the following preprocessing steps were applied:

- **Formatting & Normalization:** The flat 784-dimensional vectors were reshaped into 28×28 matrices to restore spatial context, and pixel intensities were normalized to $[0, 1]$ to stabilize training.
- **Data Augmentation:** To simulate realistic variance, the training data underwent random rotations (-15 degrees to 15 degrees) and minimal random cropping (85% to 100% scale). Horizontal flips were strictly excluded to preserve the handedness of the ASL signs.

3.3 Data Splitting and Experimental Regimes

- **Test Set:** 7,172 strictly isolated observations used exclusively for final evaluation.
- **Stratified Training Subsets:** To simulate data starvation, the training data was split into **1%, 5%, 10%, 50%, and 100%** subsets using stratified sampling to preserve exact class balances.

- **Training Regimes:** Each subset was trained twice: under a Standard Regime (raw images) and an Augmented Regime (spatial augmentations applied) to test if artificial data diversity compensates for the ViT's lack of spatial priors.

4 Model Development

4.1 CNN Baseline: Lightweight ResNet (SignLanguageCNN)

The convolutional baseline is a lightweight Residual Network (ResNet). It is constructed using a standard initial convolutional layer followed by three residual blocks.

- Inductive Bias: By utilizing 3×3 convolutional kernels, the network is forced to learn local spatial hierarchies and edge features immediately.
- Residual Connections: The shortcut connections in the basic blocks allow gradients to flow easily during backpropagation, enabling the training of deeper feature extractors without suffering from vanishing gradients.
- Efficiency: The network systematically downsamples the spatial dimensions (using stride 2) while increasing channel depth (32 to 128), finally applying Average Pooling to reduce the feature maps to a flat vector for the 25-class linear classifier.

4.2 The Sequence Model: Minimal Vision Transformer (MinimalViT)

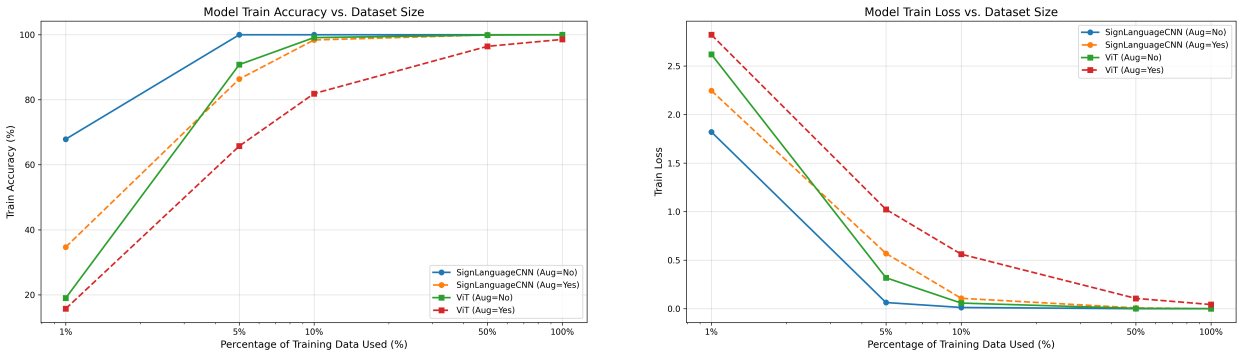
To represent models lacking built-in spatial priors, a minimal Vision Transformer was used. Instead of processing pixels locally, it treats the image as a 1D sequence of flattened patches.

- Patch Embedding: Given the small 28x28 image size, the standard 16x16 ViT patch size is too large. The PatchEmbedding layer uses a 7x7 convolutional stride to explicitly divide the image into 16 distinct tokens, projecting each into a 128-dimensional embedding space.
- Positional Encoding: Learnable 1D positional embeddings (pos_embed) are added to the tokens. The model learns the concept of 2D space entirely from scratch using these embeddings.
- Global Self-Attention: The network consists of 4 standard TransformerEncoder layers and uses multi-head self-attention to learn relationships between all 16 image patches. We utilize the pre-pended [CLS] token to generate the final classification.

5 Results

The experimental results clearly illustrate the profound impact of architectural inductive bias on sample efficiency. Across all progressive data splits, the Convolutional Neural Network (CNN) consistently outperformed the Vision Transformer (ViT) in test accuracy while utilizing approximately 40% fewer parameters and consuming less than half the peak memory. The CNN's inherent spatial priors enabled rapid generalization, achieving a robust 84.55% test accuracy with only 5% of the training data, a threshold the ViT could not reach even when exposed to 50% of the dataset. Furthermore, the efficacy of data augmentation proved highly dependent on the baseline data volume. In severely starved regimes (1% and 5%), augmentation degraded performance for both architectures, likely by introducing excessive variance before foundational patterns could be mapped. However, at the 100% data split, augmentation became a critical necessity for the ViT; without it,

the transformer severely overfit the training data (99.99% train vs. 82.15% test accuracy), but with augmentation, its test performance surged to 95.66%. This confirms that while transformers are powerful, they are heavily reliant on massive data volumes or artificially generated spatial diversity to compensate for their lack of built-in geometric assumptions.



(a) Training Accuracy

(b) Training Loss

Figure 4: Performance metrics comparing the ResNet and Vision Transformer models across the various dataset splits. (a) shows training accuracy, and (b) shows training loss.

Data Split	Model	Augmented	Train Acc (%)	Test Acc (%)	Peak Mem (MB)
1%	SignLanguageCNN	No	67.88	24.78	4.9
	SignLanguageCNN	Yes	34.67	23.26	4.9
	Minimal ViT	No	18.98	14.49	12.7
	Minimal ViT	Yes	15.69	10.40	11.9
5%	SignLanguageCNN	No	100.00	84.55	5.2
	SignLanguageCNN	Yes	86.37	76.76	5.1
	Minimal ViT	No	90.82	64.88	14.8
	Minimal ViT	Yes	65.74	56.62	14.0
10%	SignLanguageCNN	No	100.00	94.55	5.4
	SignLanguageCNN	Yes	98.40	96.04	5.1
	Minimal ViT	No	99.13	77.82	12.4
	Minimal ViT	Yes	81.89	70.02	11.6
50%	SignLanguageCNN	No	100.00	98.66	5.2
	SignLanguageCNN	Yes	99.94	99.92	4.9
	Minimal ViT	No	99.91	85.39	11.4
	Minimal ViT	Yes	96.44	89.31	10.6
100%	SignLanguageCNN	No	100.00	99.51	5.4
	SignLanguageCNN	Yes	100.00	100.00	5.1
	Minimal ViT	No	99.99	82.15	13.0
	Minimal ViT	Yes	98.57	95.66	12.1

Table 1: Comprehensive performance comparison across progressive data splits. The results demonstrate the sample efficiency of the CNN (310,009 parameters) compared to the data-hungry ViT (542,105 parameters), as well as the varying impact of data augmentation depending on available data volume.

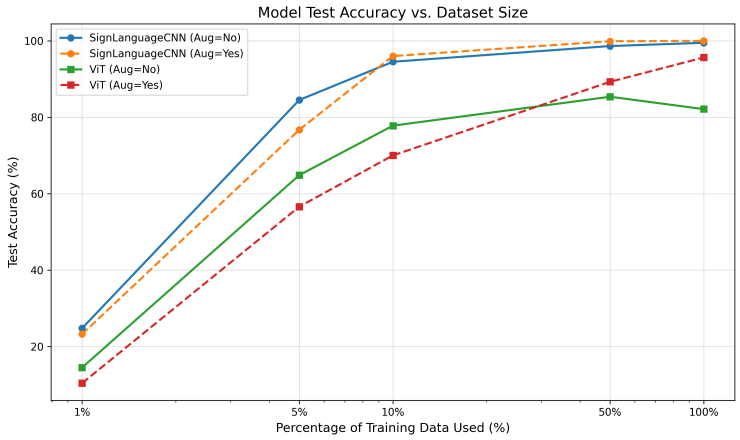


Figure 5: Test accuracy comparing the two models across various dataset splits.

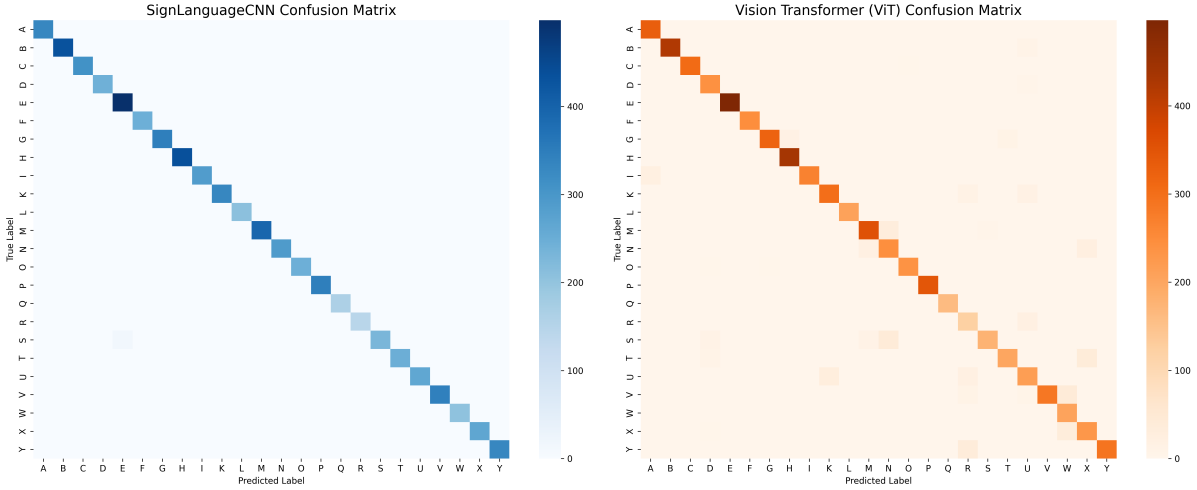


Figure 6: Comparative confusion matrices illustrating the classification precision of both models

5.1 Error Analysis

An analysis of the test set misclassifications highlights the distinct architectural behaviors of each model:

SignLanguageCNN

The CNN's confusion matrix (Figure 6) is nearly a pristine diagonal line, reflecting its exceptional precision. Its local edge-detection easily distinguished fine details, with its only notable error being the confusion of **'S'** and **'E'** (16 instances). This is highly intuitive, as both are nearly identical closed fists differentiated only by slight thumb placement.

Minimal Vision Transformer (ViT)

Conversely, the ViT's global attention mechanism struggled to resolve local features, such as explicit finger counts or precise bounding shapes, without the aid of spatial priors. Its most frequent

misclassifications included:

- **'V' misclassified as 'W' (42 instances):** The ViT failed to distinguish between two versus three extended fingers, likely blending the overlapping patch embeddings.
- **'S' misclassified as 'N' (42 instances):** Lacking localized edge detection, the transformer struggled to separate the dense pixel clusters of these similar closed-fist structures.
- **'T' misclassified as 'X' (40 instances):** Both feature a crooked index finger pointing forward; the ViT localized the general gesture but failed to accurately map the surrounding finger placements.

6 Conclusion and Future Scope

6.1 Conclusion

The experimental results of this study clearly illustrate the profound impact of architectural inductive bias on sample efficiency. Across all progressive data splits, the Convolutional Neural Network (CNN) consistently outperformed the Vision Transformer (ViT) in test accuracy while utilizing approximately 40% fewer parameters and consuming less than half the peak memory. While the CNN's inherent spatial priors enabled rapid generalization, achieving a robust 84.55% test accuracy with only 5% of the training data. Whereas, the ViT demonstrated a heavy reliance on massive data volumes or artificial augmentation to compensate for its lack of built-in geometric assumptions. Ultimately, this research confirms that while Transformers are powerful sequence models, traditional convolutional architectures remain the superior and more efficient choice for visual recognition tasks in environments where data is finite and computational resources are prioritized.

The following key takeaways summarize the performance gap observed during our experiments:

- **Sample Efficiency:** The ResNet model achieved a robust accuracy threshold with just 5% of the data (84.55%) that the ViT could not match even with 50% of the dataset (85.39%).
- **Computational Performance:** The CNN was notably more efficient, maintaining higher accuracy while utilizing significantly lower memory and parameter counts: 310,009 parameters for CNN versus 542,105 for ViT.
- **Augmentation Impact:** While augmentation was detrimental in severely starved regimes (1%–5%), it became a critical necessity for the ViT at full scale to prevent severe overfitting, raising its test accuracy from 82.15% to 95.66%.

6.2 Future Improvements

While this study provides a clear baseline, it was limited to low-resolution (28×28) grayscale images and excluded motion-dependent letters such as 'J' and 'Z'. Future research should move beyond these constraints by exploring Hybrid Architectures that combine convolutional layers for early feature extraction with Transformer blocks for global context. Additionally, investigating the role of Transfer Learning or Self-Supervised Pre-training could determine if massive external datasets allow ViTs to overcome their lack of inductive bias in data-starved environments. Expanding the dataset to include high-resolution RGB frames and temporal data for dynamic gesture recognition would also provide a more comprehensive evaluation of these models in real-world human-computer interaction scenarios

Appendix

Appendix: Github

The complete source code for this project, including the SignLanguageCNN and MinimalViT model definitions, training scripts, and the data starvation experimental pipeline, is available on GitHub. Added some on the code in the Source Code section below as well.

Repository Link: `https://github.com/hansinirajesh92/sign-language-inductive-bias`

Appendix: Source Code

Listing 1: EDA and Augmentation

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import torch
import torchvision.transforms as transforms
from PIL import Image

DATA_PATH = "Data/sign_mnist_train.csv"
OUTPUT_DIR = "output_plots"

if not os.path.exists(OUTPUT_DIR):
    os.makedirs(OUTPUT_DIR)

print("Loading dataset...")
train_df = pd.read_csv(DATA_PATH)

y_train = train_df['label'].values
x_train = train_df.drop('label', axis=1).values

def get_letter(label):
    return chr(label + 65)

print("1. Plotting Class Distribution...")
label_counts = pd.Series(y_train).value_counts().sort_index()
plt.figure(figsize=(10, 6))
plt.bar([get_letter(l) for l in label_counts.index], label_counts.values, color='sk
plt.title('Class Distribution in Sign Language MNIST')
plt.xlabel('Sign Letter')
plt.ylabel('Frequency')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.savefig(f"{OUTPUT_DIR}/01_class_distribution.png", dpi=300, bbox_inches='tight
plt.close()
```

```
x_train_images = x_train.reshape(-1, 28, 28).astype('uint8')

np.random.seed(42)
sample_indices = np.random.choice(len(y_train), 16, replace=False)

fig, axes = plt.subplots(4, 4, figsize=(8, 8))
for i, ax in enumerate(axes.flat):
    idx = sample_indices[i]
    img = x_train_images[idx]
    label = y_train[idx]

    ax.imshow(img, cmap='gray')
    ax.set_title(f"Label: {get_letter(label)}")
    ax.axis('off')
plt.tight_layout()
plt.savefig(f"{OUTPUT_DIR}/02_sample_images.png", dpi=300, bbox_inches='tight')
plt.close()

print("2. Implementing Data Augmentation Pipeline...")
augmentation_pipeline = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomRotation(degrees=(-15, 15)),
    transforms.RandomResizedCrop(size=28, scale=(0.85, 1.0), ratio=(0.9, 1.1)),
])

print("3. Plotting Original vs Augmented Examples...")
fig, axes = plt.subplots(4, 2, figsize=(6, 10))
fig.suptitle('Original vs Augmented Comparison\n(No Flips,  15  Rotation, Minima

for i in range(4):
    idx = sample_indices[i]
    original_img = x_train_images[idx]
    label = y_train[idx]

    augmented_img_pil = augmentation_pipeline(original_img)
    augmented_img = np.array(augmented_img_pil)

    axes[i, 0].imshow(original_img, cmap='gray')
    axes[i, 0].set_title(f"Original: {get_letter(label)}")
    axes[i, 0].axis('off')

    axes[i, 1].imshow(augmented_img, cmap='gray')
    axes[i, 1].set_title(f"Augmented\n(Crop & Rot  15  )", fontsize=10)
    axes[i, 1].axis('off')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(f"{OUTPUT_DIR}/03_augmented_comparison.png", dpi=300, bbox_inches='tigh
plt.close()
```

```
print("4. Plotting Standard Deviation (Variance) Images per Class...")
std_images = pd.DataFrame(x_train).groupby(y_train).std().values

fig, axes = plt.subplots(4, 6, figsize=(24, 16))
fig.suptitle('Pixel Variance (Std Dev) per Sign Language Class\n(Brighter = Higher

for i, ax in enumerate(axes.flat):
    if i < len(std_images):
        label_val = sorted(list(set(y_train)))[i]
        std_img = std_images[i].reshape(28, 28)

        im = ax.imshow(std_img, cmap='hot')
        ax.set_title(f"Var: {get_letter(label_val)}")
        ax.axis('off')
    else:
        ax.axis('off')

plt.tight_layout(rect=[0, 0.03, 1, 0.95])
plt.savefig(f"{OUTPUT_DIR}/04_variance_images.png", dpi=600, bbox_inches='tight')
plt.close()

print(f"Done! All plots saved to the '{OUTPUT_DIR}' directory.")
```

Listing 2: ResNet-based CNN Architecture

```
class BasicBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3, stride=str
        self.bn1 = nn.BatchNorm2d(out_channels)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3, stride=1
        self.bn2 = nn.BatchNorm2d(out_channels)
        self.shortcut = nn.Sequential()
        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=stride,
                nn.BatchNorm2d(out_channels)
            )
    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out += self.shortcut(x)
        return torch.relu(out)

class SignLanguageCNN(nn.Module):
    def __init__(self, num_classes=25):
        super().__init__()
        self.conv1 = nn.Conv2d(1, 32, kernel_size=3, stride=1, padding=1, bias=Fals
```

```
        self.bn1 = nn.BatchNorm2d(32)
        # 3 Residual Blocks
        self.layer1 = BasicBlock(32, 32, stride=1)
        self.layer2 = BasicBlock(32, 64, stride=2)
        self.layer3 = BasicBlock(64, 128, stride=2)
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.linear = nn.Linear(128, num_classes)
    def forward(self, x):
        out = torch.relu(self.bn1(self.conv1(x)))
        out = self.layer1(out)
        out = self.layer2(out)
        out = self.layer3(out)
        out = self.pool(out)
        return self.linear(out.flatten(1))
```

Listing 3: MinimalViT Model Definition

```
class PatchEmbedding(nn.Module):
    def __init__(self, in_channels=1, patch_size=7, emb_size=128, img_size=28):
        super().__init__()
        self.proj = nn.Conv2d(in_channels, emb_size, kernel_size=patch_size, stride
    def forward(self, x):
        x = self.proj(x)
        x = x.flatten(2).transpose(1, 2)
        return x

class MinimalViT(nn.Module):
    def __init__(self, img_size=28, patch_size=7, in_channels=1, num_classes=25, en
        super().__init__()
        self.patch_embed = PatchEmbedding(in_channels, patch_size, emb_size, img_si
        num_patches = (img_size // patch_size) ** 2
        self.cls_token = nn.Parameter(torch.randn(1, 1, emb_size))
        self.pos_embed = nn.Parameter(torch.randn(1, num_patches + 1, emb_size))
        encoder_layer = nn.TransformerEncoderLayer(d_model=emb_size, nhead=heads, d
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=depth)
        self.mlp_head = nn.Sequential(
            nn.LayerNorm(emb_size),
            nn.Linear(emb_size, num_classes)
        )
    def forward(self, x):
        B = x.shape[0]
        x = self.patch_embed(x)
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat((cls_tokens, x), dim=1)
        x = x + self.pos_embed
        x = self.transformer(x)
        return self.mlp_head(x[:, 0])
```

Listing 4: Training and Evaluation Loop

```
def train_and_eval(model_name, subset_pct, use_aug, epochs=15):
    print(f"\n--- Running: {model_name} | {subset_pct*100}% Data | Aug={use_aug} ---

    base_transforms = [transforms.ToPILImage()]
    if use_aug:
        base_transforms.extend([
            transforms.RandomRotation(degrees=(-15, 15)),
            transforms.RandomResizedCrop(size=28, scale=(0.85, 1.0), ratio=(0.9, 1.
        ])
    base_transforms.extend([transforms.ToTensor(), transforms.Normalize((0.5,), (0.
    train_transform = transforms.Compose(base_transforms)
    test_transform = transforms.Compose([transforms.ToPILImage(), transforms.ToTen

    full_train_ds = SignLanguageDataset("Data/sign_mnist_train.csv", transform=trai
    test_ds = SignLanguageDataset("Data/sign_mnist_test.csv", transform=test_transf

    if subset_pct < 1.0:
        indices, _ = train_test_split(
            np.arange(len(full_train_ds)),
            train_size=subset_pct,
            stratify=full_train_ds.labels,
            random_state=42
        )
    else:
        indices = np.arange(len(full_train_ds))

    train_ds = Subset(full_train_ds, indices)

    train_loader = DataLoader(train_ds, batch_size=128, shuffle=True, num_workers=0
    test_loader = DataLoader(test_ds, batch_size=128, shuffle=False, num_workers=0)

    model = get_resnet() if model_name == 'SignLanguageCNN' else get_vit()
    model = model.to(device)

    num_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    start_time = time.time()

    if hasattr(torch.mps, 'current_allocated_memory'):
        torch.mps.empty_cache()

    for epoch in range(epochs):
        model.train()
        for imgs, labels in train_loader:
```

```
        imgs, labels = imgs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

train_time = time.time() - start_time

peak_mem = 0
model.eval()
train_correct = 0
train_total = 0
train_loss = 0.0
with torch.no_grad():
    for imgs, labels in train_loader:
        imgs, labels = imgs.to(device), labels.to(device)
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        train_loss += loss.item()
        _, predicted = outputs.max(1)
        train_total += labels.size(0)
        train_correct += predicted.eq(labels).sum().item()

train_acc = 100. * train_correct / train_total
avg_train_loss = train_loss / len(train_loader)

model.eval()
correct = 0
total = 0
test_loss = 0.0
with torch.no_grad():
    for imgs, labels in test_loader:
        imgs, labels = imgs.to(device), labels.to(device)
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        test_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

acc = 100. * correct / total
avg_loss = test_loss / len(test_loader)

print(f"Result: Train Acc={train_acc:.2f}%, Test Acc={acc:.2f}%, Test Loss={av
return {
    'model': model_name, 'pct': subset_pct, 'aug': use_aug,
    'acc': acc, 'loss': avg_loss,
```

```
        'train_acc': train_acc, 'train_loss': avg_train_loss,
        'time': train_time,
        'memory_mb': peak_mem, 'params': num_params
    }
```

Listing 5: Error Analysis

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
import torchvision.transforms as transforms
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
import os

from benchmark_models import get_resnet, get_vit, SignLanguageDataset

device = torch.device('cuda' if torch.cuda.is_available() else 'mps' if torch.backe
print(f"Using device: {device}")

OUTPUT_DIR = "benchmark_plots"
if not os.path.exists(OUTPUT_DIR):
    os.makedirs(OUTPUT_DIR)

alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"

def train_model(model, train_loader, epochs=15):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    model.train()
    for epoch in range(epochs):
        for imgs, labels in train_loader:
            imgs, labels = imgs.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(imgs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
    return model

def get_predictions(model, test_loader):
    model.eval()
    all_preds = []
    all_labels = []
```

```
    with torch.no_grad():
        for imgs, labels in test_loader:
            imgs = imgs.to(device)
            outputs = model(imgs)
            _, preds = outputs.max(1)
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.numpy())
    return np.array(all_labels), np.array(all_preds)

def main():
    print("Loading data...")
    train_transform = transforms.Compose([
        transforms.ToPILImage(),
        transforms.RandomRotation(degrees=(-15, 15)),
        transforms.RandomResizedCrop(size=28, scale=(0.85, 1.0), ratio=(0.9, 1.1)),
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))
    ])
    test_transform = transforms.Compose([
        transforms.ToPILImage(),
        transforms.ToTensor(),
        transforms.Normalize((0.5,), (0.5,))
    ])

    train_ds = SignLanguageDataset("Data/sign_mnist_train.csv", transform=train_tra
    test_ds = SignLanguageDataset("Data/sign_mnist_test.csv", transform=test_transf

    train_loader = DataLoader(train_ds, batch_size=128, shuffle=True, num_workers=
    test_loader = DataLoader(test_ds, batch_size=128, shuffle=False, num_workers=0)

    print("Training SignLanguageCNN on 100% Data for 15 Epochs...")
    cnn = get_resnet()
    cnn = train_model(cnn, train_loader, epochs=15)
    cnn_y_true, cnn_y_pred = get_predictions(cnn, test_loader)

    print("Training ViT on 100% Data for 15 Epochs...")
    vit = get_vit()
    vit = train_model(vit, train_loader, epochs=15)
    vit_y_true, vit_y_pred = get_predictions(vit, test_loader)

    print("Generating Confusion Matrices...")
    labels_present = np.unique(cnn_y_true)
    class_names = [alphabet[l] for l in labels_present]

    fig, axes = plt.subplots(1, 2, figsize=(20, 8))

    cnn_cm = confusion_matrix(cnn_y_true, cnn_y_pred, labels=labels_present)
    sns.heatmap(cnn_cm, annot=False, cmap='Blues', ax=axes[0], xticklabels=class_na
```

```
    axes[0].set_title('SignLanguageCNN Confusion Matrix', fontsize=16)
    axes[0].set_ylabel('True Label')
    axes[0].set_xlabel('Predicted Label')

    vit_cm = confusion_matrix(vit_y_true, vit_y_pred, labels=labels_present)
    sns.heatmap(vit_cm, annot=False, cmap='Oranges', ax=axes[1], xticklabels=class_
    axes[1].set_title('Vision Transformer (ViT) Confusion Matrix', fontsize=16)
    axes[1].set_ylabel('True Label')
    axes[1].set_xlabel('Predicted Label')

    plt.tight_layout()
    plt.savefig(f"{OUTPUT_DIR}/08_confusion_matrices.png", dpi=300)
    print(f"Saved {OUTPUT_DIR}/08_confusion_matrices.png")

    def print_top_errors(cm, name):
        np.fill_diagonal(cm, 0)
        flat_cm = cm.flatten()
        top_indices = np.argsort(flat_cm)[::-1][:5]
        print(f"\nTop 5 Errors for {name}:")
        for idx in top_indices:
            r, c = divmod(idx, cm.shape[1])
            if cm[r, c] > 0:
                print(f"  True: {class_names[r]} -> Predicted: {class_names[c]} ({c

    print_top_errors(cnn_cm, "SignLanguageCNN")
    print_top_errors(vit_cm, "ViT")

if __name__ == '__main__':
    main()
```